

Spike: Task 17**Title:** Sprites and Graphics**Author:** Mohamed Shafi Uzman Fassy, 102608927**Goals / deliverables:**

The goal of this spike task is to demonstrate how I load 2d images from file and display them on the screen using SDL2.

Besides this report, what else was created?

- Code files in see COS30031-2023-102608927\17 - Spike - Sprites and Graphics\Sprite and Graphics Task\Sprite and Graphics Task

Technologies, Tools, and Resources used:

List of information needed by someone trying to reproduce this work.

- Visual Studio 2022 Community Edition
- GitHub
- SDL2 Library

Tasks undertaken:

- Project setup for SDL2 library dependencies
- Create basic bmp images.

Initialization and Image Loading:

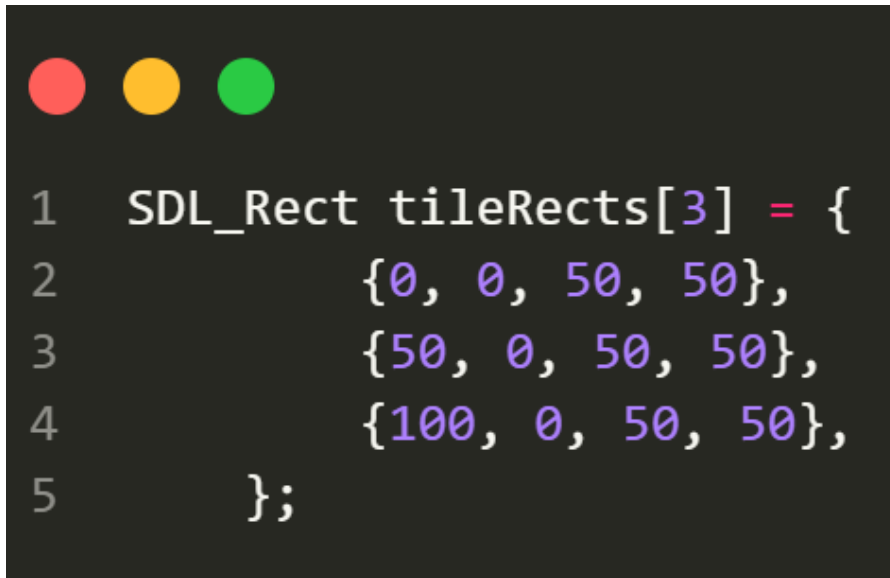
To begin the implementation, SDL's video subsystem was initialized with `SDL_Init(SDL_INIT_VIDEO)`. This is a fundamental step as it prepares SDL for any graphics-related functionality.

```
1 if (SDL_Init(SDL_INIT_VIDEO) != 0) {
2     std::cerr << "Error initializing SDL: " << SDL_GetError() << std::endl;
3     return 1;
4 }
```

Following this, I created a window using `SDL_CreateWindow`, which gives us a canvas on which graphics, such as images, can be rendered. An associated renderer was initialized using `SDL_CreateRenderer`. This renderer is pivotal as it facilitates the rendering of images onto the previously created window.

Displaying and Manipulating Images:

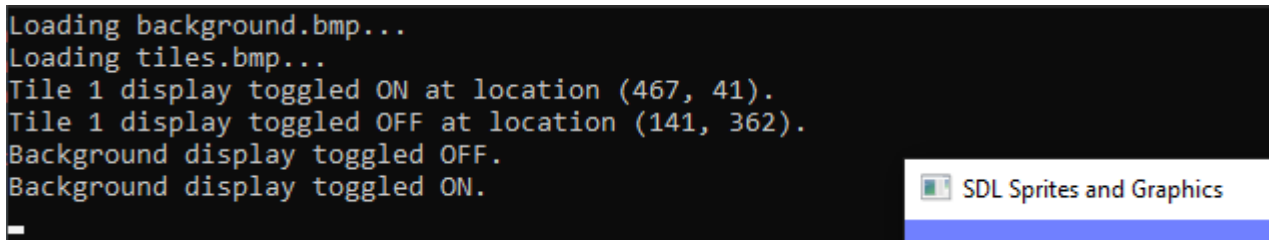
Once images are loaded, the next step is rendering them. For the background image, the renderer was used to ensure it occupied the entirety of the window. Additionally, I added a toggle using the “0”, which allows the user to make the background image visible or hidden on command.

A screenshot of a code editor with a dark background. At the top left, there are three colored circles: red, yellow, and green. The code is written in a light-colored font and defines an array of SDL_Rect structures. The code is as follows:

```
1  SDL_Rect tileRects[3] = {  
2      {0, 0, 50, 50},  
3      {50, 0, 50, 50},  
4      {100, 0, 50, 50},  
5  };
```

Using the **SDL_Rect** structure. This structure is integral to defining specific sections of the loaded tileset image. Each sub-region (or tile) was extracted from the main image and rendered onto the screen at random positions. This rendering was made interactive by allowing users to toggle the visibility of each tile using the number keys. This is evident in the case structures where specific keys correspond to displaying specific tiles.

Debugging:

A screenshot of a console window with a black background and white text. The text shows the following sequence of messages:

```
Loading background.bmp...  
Loading tiles.bmp...  
Tile 1 display toggled ON at location (467, 41).  
Tile 1 display toggled OFF at location (141, 362).  
Background display toggled OFF.  
Background display toggled ON.
```

On the right side of the console window, there is a small icon of a document with a green checkmark, followed by the text "SDL Sprites and Graphics".

By using console logs, it was very instrumental when I was debugging. As the embedded messages like “loading image1.bmp” or “tile 3 display ON at location (10, 40)”, tracking the application's flow and diagnosing issues became considerably more manageable. Furthermore, it's important to highlight the concept of optimizing images to the current

screen's bit-depth. This optimization ensures that the rendering process is efficient, minimizing potential performance lags or visual artifacts.